

Instructions

Vishal Bhatoy

January 2026

We provide instructions for working with 4-dimensional Koszul Artin-Schelter regular algebras in the computer algebra system Magma. All algebras are listed in the file `Algebras.m` and we provide code to compute the superpotential as well as the discrete invariants for each partition $\lambda \vdash 4$.

1 Creating Algebras

We define graded algebras in Magma by specifying a set of generators and homogeneous relations in the free algebra over a given base field. Let $\mathbb{K}$ be a base field, and consider generators $\{x_1, \dots, x_n\}$, each of degree 1, along with a set of homogeneous relations $\{r_1, \dots, r_m\}$. Mathematically, such an algebra is given by:

$$A = \frac{\mathbb{K}\langle x_1, \dots, x_n \rangle}{(r_1, \dots, r_m)},$$

where $\mathbb{K}\langle x_1, \dots, x_n \rangle$ denotes the free algebra over $\mathbb{K}$, and the ideal of relations is generated by the r_i . In Magma, the construction proceeds by first defining the base field and free algebra:

```
// Define base field
K := <base field>;

// Free algebra on n generators
F<x1, x2, ... , xn> := FreeAlgebra(K, n);
Gens := [x1, x2, ... , xn];

// Then define the list of relations:
Rels := [r1, r2, ... , rm];
```

Let us now consider a specific example, namely Algebra M. We work over the rational function field $\mathbb{K} = \mathbb{Q}(f, h)$, and define a free algebra on generators x, y, z, w , each of degree 1. The Algebra M is then defined by the following six quadratic relations:

```
// Define the base field and a function field with parameters f and h
QQ := RationalField();
K<f,h> := FunctionField(QQ, 2);

// Define the free algebra with generators x, y, z, w
F<x,y,z,w> := FreeAlgebra(K,4);
Gens := [x,y,z,w];

// Define the six quadratic relations
Rels := [x*w + w*x,
         z*y + y*z,
         y*w + h*(-x*y - w*z ),
         y*x + h*(-f*w*y + x*z ),
         z*w + h*(-w*y + x*z ),
         z*x + h*(x*y + f*w*z ) ];
```

2 Algebras.m File

The file `Algebras.m` contains the defining relations for Artin-Schelter regular algebras considered in this project. These algebras are taken from the literature and represent several known families of Koszul algebras with quadratic relations.

Each algebra is specified by:

- a base field $\mathbb{K}$
- a free algebra $F = \mathbb{K}\langle x, y, z, w \rangle$ on four generators of degree 1;
- a list of six quadratic relations that define the algebra.

Each algebra is stored as a tuple `<K, F, Rels, Gens>` (base field, free algebra, relations, generators) in a named variable like `AlgebraA`, `AlgebraB`, and so on. These variables can then be used for further computations (e.g. computing the superpotential and discrete invariants). Each algebra has different conditions on the parameters h , f or p . We have defined all 77 known families of algebras in this way. Some examples are given below:

```
// Algebra A
<Univariate rational function field over Rational Field
Variables: h, Free associative algebra of rank 4 over Univariate rational function field over Rational Field
Order: Non-commutative Graded Lexicographical
Variables: x, y, z, w,
[-x*w + w*x,
-y^2 - y*z + z*y,
y*w - h*w*y,
-h*x*y + y*x - h*w*z,
z*w - h*w*z,
2*h*x*y - h*x*z + z*x + h*w*z],
[x,
y,
z,
w]>

// Algebra M
<Multivariate rational function field of rank 2 over Rational Field
Variables: h, f, Free associative algebra of rank 4 over Multivariate rational function field of rank 2 over Rational Field
Order: Non-commutative Graded Lexicographical
Variables: x, y, z, w,
[x*w + w*x,
y*z + z*y,
-h*x*y + y*w - h*w*z,
h*x*z + y*x - h*f*w*y,
h*x*z + z*w - h*w*y,
h*x*y + z*x + h*f*w*z],
[x,
y,
z,
w]>
```

3 Superpotential.m File

The `Superpotential.m` code takes a basis of the vector space V and the quadratic relations $R \subseteq V \otimes V$ as input. It builds subspaces $S_0, S_1, S_2 \subseteq V^{\otimes 4}$ using tensor operations and computes their intersection $S_0 \cap S_1 \cap S_2$, which is one-dimensional. From this intersection, the code extracts a generator $\omega \in V^{\otimes 4}$ representing the superpotential, which is unique up to a scalar. We now compute the superpotential for Algebra M. The code is shown below:

```
// Generate monomials of degrees 2, 3, and 4 in the free algebra
m2 := [B*A: A in Gens, B in Gens];
m3 := [B*A: B in m2, A in Gens];
m4 := [B*A: B in m3, A in Gens];

Construct the subspace R tensor V tensor V.
RA1A1 := [i*j*k: i in Rels, j in Gens, k in Gens];
RtensorA1tensorA1 := Matrix([[MonomialCoefficient(1,k): k in m4] : 1 in RA1A1]);

// Construct the subspace V tensor V tensor R
A1A1R := [i*j*k: i in Gens, j in Gens, k in Rels];
A1tensorA1tensorR := Matrix([[MonomialCoefficient(1,k): k in m4] : 1 in A1A1R]);

// Construct the subspace V tensor R tensor V
A1RA1 := [i*j*k: i in Gens, j in Rels, k in Gens];
A1tensorRtensorA1 := Matrix([[MonomialCoefficient(1,k): k in m4] : 1 in A1RA1]);

// Compute the intersection
spb := Basis(Image(RtensorA1tensorA1) meet Image(A1tensorA1tensorR) meet Image(
    A1tensorRtensorA1));

// Calculate superpotential
wsup := &+[sp[i]*m4[i]: i in [1..256]];
MC := Vector([MonomialCoefficient(wsup,k): k in m4]);
```

Using the Magma code above, the superpotential ω is defined as

$$\begin{aligned} \omega := & (h^2f - h^2)xyz + (hf - h)xyzw + (-h^2f + h^2)xywy + (-h^2f + h^2)xzxy + (hf - h)xzyw + (-h^2f + \\ & h^2)xzww + (-h^3f^2 + 2h^3f - h^3)xwyz + (-h^3f^2 + 2h^3f - h^3)xwzy - yxyw - yxxz + (-hf + h)yxwz - \\ & 1/hyzxw - 1/hyzwx + (-hf + h)ywxz + ywyx - fywzw + zxyx - zxxw + (-hf + h)zxwy - 1/hzyxw - \\ & 1/hzywx + (-hf + h)zwx y + fzw yw + zwzx + (-h^3f^2 + 2h^3f - h^3)wxyz + (-h^3f^2 + 2h^3f - h^3)wxzy + \\ & (h^2f - h^2)wyxy + (hf - h)wyzx + (h^2f^2 - h^2f)wywz + (h^2f - h^2)wzxx + (hf - h)wzyx + (-h^2f^2 + h^2f)wzwy. \end{aligned}$$

We can compute the superpotential ω directly for any algebra in the `Algebras.m` file using the function we wrote, for example:

```
superpotential(AlgebraA);
superpotential(AlgebraB);
superpotential(AlgebraM);
// and so on...
```

4 Idempotents.m File

After obtaining the superpotential, we decompose the tensor space $V^{\otimes 4}$ under the action of the symmetric group S_4 . This allows us to extract the component of the superpotential corresponding to each irreducible representation indexed by a partition $\lambda \vdash 4$. The `Idempotents.m` file constructs all 24 elements of S_4 as permutation matrices acting on $V^{\otimes 4}$, and computes the idempotents $e_\lambda \in \mathbb{K}[S_4]$ for each partition $\lambda \vdash 4$.

```
MC := Vector([MonomialCoefficient(wsup,k): k in m4]);
A12 := Matrix(K, 16,16,
[1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0, 0,0,0,0,1,0,0,0,0,0,0,0,0,0,0,0,
0,0,0,0,0,0,0,0,1,0,0,0,0,0,0,0, 0,0,0,0,0,0,0,0,0,0,0,0,1,0,0,0,
0,1,0,0,0,0,0,0,0,0,0,0,0,0,0,0, 0,0,0,0,0,1,0,0,0,0,0,0,0,0,0,0,
0,0,0,0,0,0,0,0,0,0,0,1,0,0,0,0,0, 0,0,0,0,0,0,0,0,0,0,0,0,0,1,0,0,
0,0,1,0,0,0,0,0,0,0,0,0,0,0,0,0, 0,0,0,0,0,0,0,1,0,0,0,0,0,0,0,0,
0,0,0,0,0,0,0,0,0,0,0,0,1,0,0,0,0, 0,0,0,0,0,0,0,0,0,0,0,0,0,0,1,0,
0,0,0,1,0,0,0,0,0,0,0,0,0,0,0,0, 0,0,0,0,0,0,0,0,1,0,0,0,0,0,0,0,
0,0,0,0,0,0,0,0,0,0,0,0,0,1,0,0,0, 0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1]);

I16:= IdentityMatrix(K, 16);
p34:= KroneckerProduct(A12,I16);
p12:= KroneckerProduct(I16,A12);
I4 := Matrix(K, 4,4, [1,0,0,0, 0,1,0,0, 0,0,1,0, 0,0,0,1]);
KP:= KroneckerProduct(I4,A12);
p13:= KroneckerProduct(KP,I4);
p123:= p13*p12;
p1234:= p123*p34;
Z:= p34*p12;
p23:= p123*p13;
p234:= p12*p1234;
p243:= p34*p23;
p24:= p23*p243;
p1243:= p243*p13;
p124:= p34*p1234;
p132:= p12*p13;
p1342:= p123*p1234;
p134:= p23*p1234;
T:= p24*p13;
p1324:= p124*p13;
p1432:= Z*p13;
p142:= p134*T;
p143:= p12*p1432;
p14:= p123*p1432;
p1423:= p34*T;
Q:= p14*p23;
e := KroneckerProduct(I16, I16);

// Corresponding to partition (4)
eV400 := (1/24) * (e + p12 + p13 + p14 + p23 + p24 + p34 + p123 + p132 + p124 +
p142 + p134 + p143 + p234 + p243 + p1234 + p1243 + p1342 + p1324 + p1423 +
p1432 + Z + T + Q);
eV400w := MC * eV400;

// Corresponding to partition (1,1,1,1)
eV000 := (1/24) * (e + (-1)*(p12 + p13 + p14 + p23 + p24 + p34) + (1)*(p123 + p132
+ p124 + p142 + p134 + p143 + p234 + p243) + (-1)*(p1234 + p1243 + p1342 +
p1324 + p1423 + p1432) + (1)*(Z + T + Q));
eV000w := MC * eV000;
```

```

// Corresponding to partition (3,1)
eV210:= (3/24) * (3*e + (1)*(p12 + p13 + p14 + p23 + p24 + p34) + (0)*(p123 + p132
+ p124 + p142 + p134 + p143 + p234 + p243) + (-1)*(p1234 + p1243 + p1342 +
p1324 + p1423 + p1432) + (-1)*(Z + T + Q));
eV210w := MC * eV210;

// Corresponding to partition (2,1,1)
eV101:= (3/24) * (3*e + (-1)*(p12 + p13 + p14 + p23 + p24 + p34) + (0)*(p123 +
p132 + p124 + p142 + p134 + p143 + p234 + p243) + (1)*(p1234 + p1243 + p1342 +
p1324 + p1423 + p1432) + (-1)*(Z + T + Q));
eV101w:= MC*eV101;

// Corresponding to partition (2,2)
eV020:= (2/24) * (2*e + (0)*(p12 + p13 + p14 + p23 + p24 + p34) + (-1)*(p123 +
p132 + p124 + p142 + p134 + p143 + p234 + p243) + (0)*(p1234 + p1243 + p1342 +
p1324 + p1423 + p1432) + (2)*(Z + T + Q));
eV020w:= MC*eV020;

```

5 Computation of Discrete Invariants

We now turn to the computation of discrete invariants for each partition $\lambda \vdash 4$, using the superpotential and idempotents defined above. The partition $(1,1,1,1)$ does not help distinguish between the algebras, so we are not discussing it here.

5.1 Invariants for the partition $\lambda = (4)$

As $\lambda = (4)$, the partial flag variety reduces to the projective space $\mathbb{P}^3$. We begin by constructing this projective space over the rational function field $\mathbb{Q}(h, f)$, taking the base field as a function field in two variables. The code for Algebra M is given below.

```

QQ := RationalField();
K<f,h> := FunctionField(QQ, 2);
P<x,y,z,w> := ProjectiveSpace(K, 3);
X := Scheme(P, eV400w);

```

We then compute the discrete invariants of the scheme X , as follows:

```

// Primary components of the scheme X
[[Dimension(p), Degree(p)] : p in PrimaryComponents(X)];
> [ [ 2, 1 ], [ 2, 1 ], [ 2, 1 ], [ 2, 1 ] ]

// Primary components of the reduced subscheme of X
[[Dimension(p), Degree(p)] : p in PrimaryComponents(ReducedSubscheme(X))];
> [ [ 2, 1 ], [ 2, 1 ], [ 2, 1 ], [ 2, 1 ] ]

// Primary components of the singular locus of X
[[Dimension(p), Degree(p)] : p in PrimaryComponents(SingularSubscheme(X))];
> [ [ 1, 1 ], [ 1, 1 ], [ 1, 1 ], [ 1, 1 ], [ 1, 1 ], [ 1, 1 ] ]

// Primary components of the reduced singular locus
[[Dimension(p), Degree(p)] : p in PrimaryComponents(ReducedSubscheme(
SingularSubscheme(X)))]];
> [ [ 1, 1 ], [ 1, 1 ], [ 1, 1 ], [ 1, 1 ], [ 1, 1 ], [ 1, 1 ] ]

```

5.2 Invariants for the partitions $\lambda = (2, 2), (2, 1, 1), (3, 1)$

We compute the discrete invariants corresponding to the partitions $\lambda = (2, 2), (2, 1, 1), (3, 1)$ using the Magma functions we have developed, as described below.

1. Load the algebra:

First, we need to load the algebra we want to study. For example, to compute invariants for Algebra M, we load it from `Algebras.m` as a tuple `<K, F, Rels, Gens>`. All algebras are defined as tuples in `Algebras.m`, as explained in Section 2.

2. Run the discrete invariants function:

- For the partition $(2, 2)$, we use the function `DiscreteInvariantsX020`
e.g. `DiscreteInvariantsX020(AlgebraM);`
- For the partition $(2, 1, 1)$, we use the function `DiscreteInvariantsX101`
e.g. `DiscreteInvariantsX101(AlgebraM);`
- For the partition $(3, 1)$, we use the function `DiscreteInvariantsX210`
e.g. `DiscreteInvariantsX210(AlgebraM);`

3. Output:

Each function directly computes and returns the discrete invariants for the algebra, which are the dimensions and degrees of the primary components of the scheme, its reduced subscheme, its singular subscheme, and the reduced subscheme of the singular subscheme.

Note. For the partitions $(2, 1, 1)$ and $(3, 1)$, we define the base field differently because the code requires a fourth root of unity, i , satisfying $i^4 = 1$. This is used in defining the idempotents e_{L_1} and e_{L_3} .

```
// Define base field with fourth root of unity
QQ := RationalField();
P<p> := PolynomialRing(QQ);
N<i> := NumberField(p^2 + 1);
RR<s> := PolynomialRing(N);
K<h> := FunctionField(RR);

// Define free algebra on four generators
F<x,y,z,w> := FreeAlgebra(K,4);

// e, u1, u2, u3 are elements of the cyclic group C4

// Define the idempotents
eL0 := (1/4)*(e + u1 + u2 + u3);
eL1 := (1/4)*(e + i*u1 - u2 - i*u3);
eL2 := (1/4)*(e - u1 + u2 - u3);
eL3 := (1/4)*(e - i*u1 - u2 + i*u3);
```